



BSc (Hons) in **Information
Technology Specialized in AI**
**IT3012 : Intelligent
Agents**

Faculty of Computing

Practical 04

Year 3 · Semester 1 · 2026

Semester 2

Objective & Lecture Mapping: In previous labs, you explored Uninformed Search strategies like BFS and UCS inside `agent.py` [cite: 1]. Today, we transition to **Informed Search**. You will enhance your agent by implementing the **A* Search** algorithm, using **Heuristics** to drastically reduce the number of explored nodes in the `visual_grid_game.py` environment [cite: 4]. You will start with basic heuristic functions and connect them to your search logic.

Part 1: Practical Implementation — Building the A* Agent

Step 1.1: Implementing the Heuristic Functions

Informed search algorithms require a heuristic function, $h(n)$, which estimates the cheapest path from the current node to the goal. You will calculate distance metrics on a 2D grid.

1. Create a new Branch called `Week_04` in your Forked Github Repo.
2. Open `agent.py` and locate your `SearchAgent` class [cite: 1].
3. Add a new method named `def manhattan_distance(self, pos, goal):` that calculates the distance using the formula $h(n) =$

$|x_1 - x_2| + |y_1 - y_2|$ and returns the integer value.

4. Add a second method named `def euclidean_distance(self, pos, goal):` that calculates the straight-line distance using the formula $h(n) = \sqrt{(x_1 - x_2)^2 + (y_1 - y_2)^2}$. You will need to `import math` for this.
5. *Testing Checkpoint:* Print the outputs of both functions for a mock start position (0, 0) and goal (3, 4) to verify that Manhattan returns 7 and Euclidean returns 5.0.

Step 1.2: Implementing A* Search

A* evaluates nodes by combining the path cost so far, $g(n)$, and the estimated cost to the goal, $h(n)$. The evaluation function is $f(n) = g(n) + h(n)$.

1. Inside `SearchAgent`, create a new method: `def astar_search(self, start_pos, goal_pos, walls, grid_size, heuristic_type='manhattan') .`
2. Initialize an empty priority queue using `heapq` and an empty set for `reached_states` .
3. Push the initial state into the priority queue. Unlike UCS which only uses $g(n)$, your A* tuple must be formatted as: `(f_cost, g_cost, current_pos, path_taken)` . For the starting node, $g(n) = 0$. Calculate $h(n)$ using your chosen heuristic method, and set $f(n) = g(n) + h(n)$.
4. Create your standard `while` loop to process the queue. Pop the node with the lowest `f_cost` . If `current_pos == goal_pos` , return the `path_taken` . Otherwise, add `current_pos` to `reached_states` .
5. For node expansion, check the four adjacent cells (Up, Down, Left, Right). For each valid neighbor (not a wall, within bounds, and not reached): Calculate $g_{\text{new}} = g_{\text{current}} + 1$, h_{new} using your

heuristic, and $f_{\text{new}} = g_{\text{new}} + h_{\text{new}}$. Push the new tuple to the priority queue.

Step 1.3: Integrating A* into the Agent's Decision Loop

Finally, you need to connect your new search algorithm to the agent's perception and action loop so it can run in the visual environment.

1. Navigate to the `sense_and_act(self, percept)` method in your `SearchAgent`.
2. Add an `elif self.active_algo == 'AStar':` block to handle the new algorithm alongside your existing BFS/DFS/UCS.
3. Extract the necessary global state from the percept dictionary (e.g., `percept['remaining_food']`).
4. Find the closest food item to act as the `goal_pos`. Call your `astar_search` method and store the returned list of actions in `self.plan`.
5. Open `visual_grid_game.py` and modify the `GridGameGUI` initialization to inject your `SearchAgent()` instead of the `ModelBasedAgent()`. Run the simulation and observe the optimized pathfinding!

Part 2: Theoretical Evaluation

Based on your implementation and Lecture 05, complete the following analytical questions.

1. (Understand) What is the key difference between how Uniform-Cost Search (UCS) and A* Search prioritize which node to explore

next?

- **Uniform-Cost Search (UCS)** prioritizes nodes strictly by their accumulated path cost from the start node:

$$f(n) = g(n)$$

UCS expands nodes blindly in concentric **cost contours** in all directions, without any awareness of where the goal is located.

- A *Search** prioritizes nodes using both the actual cost incurred so far, **$g(n)$** , and an estimated remaining cost to the goal, **$h(n)$** :

$$f(n) = g(n) + h(n)$$

By adding the heuristic **$h(n)$** , A* directs its search toward the goal, expanding far fewer unpromising nodes compared with UCS.

2. (Analyze) In Step 1.1, you used Manhattan Distance. Why is Manhattan Distance considered an "admissible" heuristic for this specific 4-way movement grid, and what would happen to your A* algorithm if the heuristic was NOT admissible?

- **Why Manhattan Distance is Admissible:**

An admissible heuristic never overestimates the true minimum cost to reach the goal:

$$h(n) \leq h(n)^*$$

On a 4-way grid (Up, Down, Left, Right, with unit cost 1), the shortest possible distance between **(x_1, y_1)** and **(x_2, y_2)** without obstacles is:

$$|x_1 - x_2| + |y_1 - y_2|$$

Adding walls can only increase or maintain the actual path length **$h(n)^*$** . Therefore:

$$h(n) \leq h(n)^*$$

always holds.

- **Consequence of Non-Admissible Heuristics:**

If **$h(n)$** overestimates the actual cost:

$$h(n) > h(n)^*$$

then A^* is **no longer guaranteed to find the optimal (shortest) path**. The algorithm might select a suboptimal path because it over-penalizes the true optimal path with an artificially inflated $f(n)$ value.

3. (Evaluate) If we modified `visual_grid_game.py` to allow the agent to move diagonally (8-way movement), would Manhattan distance still be an admissible heuristic? Why or why not? Which metric should you switch to?

- **No.**

- **Why?**

In an 8-way grid with unit-cost diagonal moves, moving from **(0, 0)** to **(1, 1)** requires **1 diagonal step**, so the actual cost is:

$$h(n) = 1^*$$

However, Manhattan Distance calculates:

$$|1 - 0| + |1 - 0| = 2$$

Because:

$$2 > 1$$

Manhattan Distance **overestimates** the true cost, violating the admissibility condition:

$$h(n) \leq h(n)^*$$

- **Which metric should be used instead?**

Chebyshev Distance — if diagonal steps cost 1:

$$h(n) = \max(|x_1 - x_2|, |y_1 - y_2|)$$

Octile Distance — if diagonal steps cost $\sqrt{2} \approx 1.414$:

$$h(n) = \Delta x + \Delta y + (\sqrt{2} - 2) \times \min(\Delta x, \Delta y)$$

where:

$$\Delta x = |x_1 - x_2|$$

$$\Delta y = |y_1 - y_2|$$

4. (Create) When targeting multiple food items simultaneously, calculating the distance to just the single closest food item is a weak heuristic. Propose (in text) a stronger heuristic for navigating the grid to eat ALL remaining food efficiently.

Calculating the distance to only the single closest food item ignores the remaining food locations and can lead to inefficient backtracking or **local greedy traps**.

Proposed Stronger Heuristic: Minimum Spanning Tree (MST)

1. **Construct a Graph:** Treat the current agent position and all remaining food pellets as vertices.
2. **Calculate Edge Weights:** Use the pairwise Manhattan Distance between every pair of vertices as the edge weights.
3. **Compute the Minimum Spanning Tree (MST):** Use **Prim's algorithm** or **Kruskal's algorithm** to calculate the weight of the Minimum Spanning Tree connecting all uncollected food items.
4. **Calculate the Heuristic Value:**

$$h(n) = \text{Manhattan distance}(\text{agent, closest food}) + \text{weight}(\text{MST of remaining food items})$$

- **Why it is effective:**

The MST weight represents a **lower bound** on the path cost required to connect and visit all remaining food items. Therefore, this heuristic is **admissible** because it does not overestimate the true remaining cost, while also providing stronger global guidance for minimizing the overall path length needed to collect all food.

Once Completed Get a PDF of the completed File and Push into the Week 04 branch in the Github Project

Finalize and Lock Lab 04 Documentation

Incomplete: {filledCount}/{fields.length} questions answered. Please provide detailed responses for all questions.



FACULTY OF COMPUTING